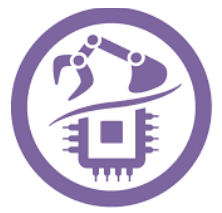




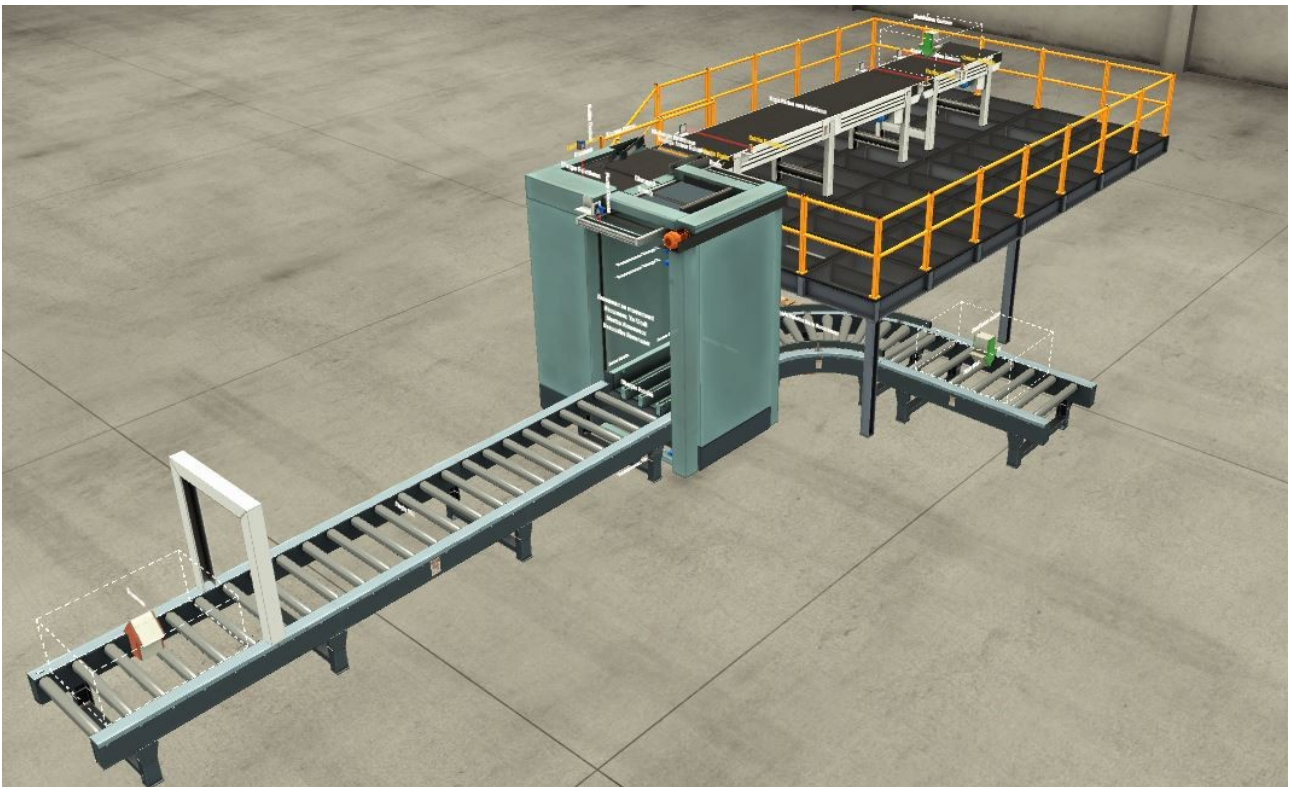
POLYTECH[®]
MONTPELLIER



MEA
Microélectronique
et automatique

Systèmes à évènements discrets

Automatisation d'un palettiseur



Raphaël Dias-Coelho
Antonin Delbouys

MEA 4

Table of Contents

1 Introduction.....	1
2 Structure du code.....	1
2.1 Actionneurs et capteurs.....	1
2.2 Structure générale du main.c.....	2
2.3 Les « switch ».....	2
2.3.1 Entité 1 : distribution de cartons	3
2.3.2 Entité 2: dé/blocage entrée palettiseur.....	5
2.3.3 Entité 3 : poussoir cartons.....	7
2.3.4 Entité 4: Porte	8
2.3.5 Entité 5: Ascenseur.....	10
3 Annexe.....	12

1 Introduction

L'objectif de ce projet est d'implémenter un palettiseur afin de le rendre autonome, à l'aide du logiciel Factory IO ainsi qu'une STM32.

La première étape a donc été de bien comprendre les mécanismes du système et de le séparer en six grandes entités autonomes. Cela nous a donc permis de construire le réseau de Pétri du système.

Après avoir réalisé notre réseau de Pétri, nous avons dû le traduire en langage C sur une STM32 afin d'automatiser nous-même le palettiseur.

Pour cela, nous avons utilisé une structure en « switch » pour notre code. Cette structure nous a permis de bien différencier les six entités entre elles.

2 Structure du code

2.1 Actionneurs et capteurs

Tout notre projet tourne autour de deux éléments : lire l'état de nos capteurs et activer ou désactiver nos actionneurs. En effet, ce sont ces deux phénomènes qui vont faire circuler nos cartons et observer leurs avancés dans le palettiseur.

Pour lire l'état des capteurs, nous utilisons la fonction « `FACTORY_IO_Sensors_Get` » implémentée dans le fichier *factory_io.c*. Celle-ci va lire l'état des capteurs aux moments choisis parmi la liste des capteurs définie dans le fichier *main.h*. En fonction de l'état allumé ou éteint des capteurs, on va activer ou éteindre des actionneurs.

Afin de donner l'état que l'on désire à nos actionneurs, on utilise la fonction « `FACTORY_IO_Actuators_Set` » implémentée également dans le fichier *factory_io.c*. Nous utilisons la liste des actionneurs détaillée dans le fichier *main.h* comme pour les capteurs. Dans la liste des capteurs et actionneurs nous n'utilisons pas les bits 7 et 15, car ils sont utilisés dans d'autres fonctions.

Pour les actionneurs, nous utilisons un buffer de 32 bits, chaque bit (les 21 premiers) correspond à un actionneur. À l'aide des opérateurs « `|=` » ou encore « `& =~` », on peut respectivement allumer ou éteindre des bits de notre *buffer_Actuator*.

2.2 Structure générale du main.c

Le *main.c* commence par l'instanciation de toutes les variables que l'on va utiliser dans la boucle infinie. On a par exemple les « State_entity ». Chaque entité possède sa propre variable « State_entity », qui sera la variable (le « case ») dans son « switch ». Cette variable est initialisée à 1 pour montrer que nous sommes dans le premier état de l'entité. Par exemple, « State_entity_1 » pour la première entité. La valeur de cette variable correspond à l'état dans lequel on se trouve dans le réseau de Pétri.

Ensuite, on trouve les variables relatives aux synchronisations, aux compteurs ainsi que pour les anneaux mémoires. La première entité utilise en effet un anneau mémoire, que l'on expliquera plus tard dans le rapport. Les synchronisations quant à elles font les liens entre les entités. En effet, les entités requièrent parfois une action réalisée par des actionneurs ou capteurs qui appartiennent à une autre entité pour passer d'un état à un autre.

Toutes ces variables représentent chaque type de liaison entre les états ainsi qu'entre les entités dans le réseau de Pétri. Notre but dans le code étant de retranscrire exactement la même chose que le schéma mais dans le langage C. On passe ensuite dans la boucle infinie, où la traduction du réseau de Pétri se trouve.

2.3 Les « switch »

Pour commencer, on éteint tous les actionneurs exceptés les tapis ainsi que le « blocage entrée palettiseur », qui est la porte entre le premier tapis et le poussoir. Cette action va faire en sorte que tous les tapis soient en marche tout le temps. Pour la porte, on la remonte au début afin de bloquer les deux premiers cartons qui vont arriver. Nous la baisserons dans la suite du code.

2.3.1 Entité 1 : distribution de cartons

Nous commençons donc ici le codage de la première machine à état avec notre premier switch : « switch (State_entity_1) ». Ensuite, chaque case correspond à un état (un « rond » du réseau de Pétri) :

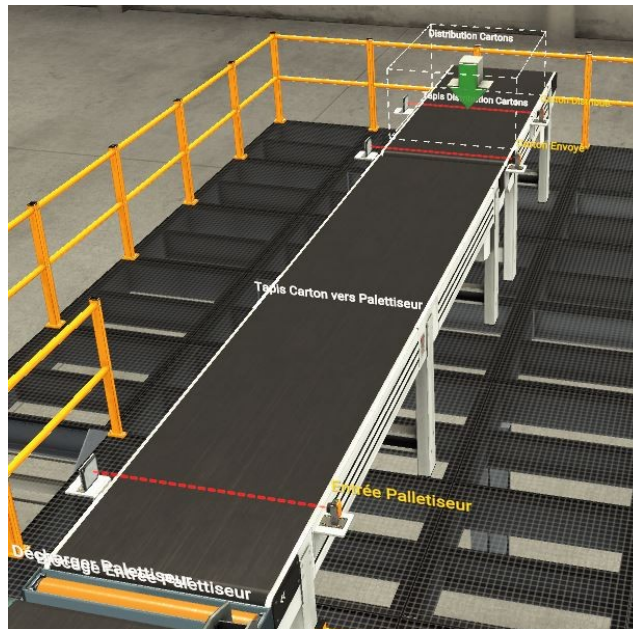


Figure 1: Entité 1 palettiseur

- CASE 1

Dans ce premier case, nous allumons l'actionneur de distribution de carton grâce à la commande « buffer_Actuator |= distrib_c ». Cela va permettre d'envoyer un carton sur le premier tapis (« tapis_distrib_c »).

```
switch (State_entity_1)
{
    case 1: // distrib_c
        buffer_Actuator |= distrib_c;
        if (FACTORY_IO_Sensors_Get(c_distribue) == 0)
        {
            State_entity_1 ++;
        }
        break;
}
```

On va ensuite vérifier que le carton a bien été envoyé en regardant l'état du capteur « c_distribue ». Ces capteurs sont allumés (état 1) lorsqu'ils captent le faisceau laser de l'autre côté du tapis. Le capteur observé va donc passer à 0 lorsque le carton envoyé va passer devant.

Quand ce processus arrive, nous rentrons dans le « if (FACTORY_IO_Sensors_Get(c_distribue) == 0) » et nous pouvons donc passer à l'état suivant en incrémentant de 1 la variable *State_entity_1*.

- CASE 2

Ici, après avoir vérifié qu'un carton est bien parti, nous arrêtons la distribution de carton en éteignant l'actionneur en question :

```
case 2://envoi carton
    buffer_Actuator &= ~distrib_c;
    if (FACTORY_IO_Sensors_Get(c_envoye) == 0)
    {
        State_entity_1 ++;
    }
    break;
```

« buffer_Actuator &= ~distrib_c ». Ceci nous évite d'avoir une accumulation de cartons devant le blocage palettiseur et ainsi mieux gérer le flux de cartons.

Afin de vérifier que notre premier carton avance bien sur le tapis, on vérifie que celui-ci passe devant le capteur « c_envoye » et donc passe à l'état 0. Si tel est le cas, on passe à l'état suivant.

- CASE 3

Ce case représente un anneau mémoire qui va nous permettre d'attendre d'avoir 2 cartons devant l'entrée du blocage palettiseur avant de passer à l'entité suivante.

```
case 3: //attente
    if (A_Premier_C == 1 && A_Deuxieme_C == 0)
//attente 1er carton
    {
        State_entity_1 = 1;

        A_Premier_C = 0;
        A_Deuxieme_C = 1;
    }

    else if (A_Premier_C == 0 && A_Deuxieme_C == 1)
//attente 2eme carton
    {
        Synch_1_2 = 1;

        A_Premier_C = 1;
        A_Deuxieme_C = 0;

        State_entity_1 ++;
    }
    break;
```

On retranscrit donc en C le réseau de pétri. Avec des constantes prédéfinies avant

de rentrer dans le *main()*, lorsque A_premier_C vaut 1 et A_deuxieme_C vaut 0, on est dans le cas où l'on attend un premier carton. Lorsque ce dernier est arrivé alors on a A_premier_C qui passe à 0 et A_deuxieme_C qui passe à 1 jusqu'à ce qu'un second carton arrive et dans ce cas on passe au case suivant. Nous utilisons aussi la première synchronisation, qui permettra de commencer la deuxième machine à état, quand le prochain case, d'attente, sera terminé.

- CASE 4

Le quatrième et dernier état de cette première entité fait apparaître une attente. Pour recommencer au début de l'entité 1,

```
case 4: // wait
    if (Synch_3_1)
    {
        Synch_3_1=0;
        State_entity_1 = 1;
    }
    break;
```

nous devons en effet attendre un événement venant de l'entité 3 (que la butée carton soit à 0 avant de renvoyer des cartons). Quand cet événement de l'entité 3 passe à 1, nous rentrons donc dans le « if ». Quand cet événement se déroule, on remet l'attente à l'état 0 et on recommence le déroulement de la première entité en remettant la variable State_entity_1 à la valeur 1.

Ici, on a supposé que le renvoi de cartons à lieu lorsque le capteur « butée_carton » détecte des cartons, d'où cette attente avec la synchronisation par rapport à l'entité 3.

Ceci montre donc la fin de la première entité.

2.3.2 Entité 2: dé/blocage entrée palettiseur

Cette deuxième entité représente le mécanisme qui fait rentrer dans la partie poussoir les deux cartons distribués par la première entité :



Figure 2: Entité 2 palettiseur

- CASE 1

Dans ce premier case on va vérifier si les 2 cartons ont bien été envoyés sur les tapis. Pour ce faire, on vérifie si la synchronisation entre l'entité 1

et 2 est bien passée à 1. Par ailleurs, si le capteur "entree_palettiseur" ne capte plus(=0), (i.e il y a un carton devant l'entrée du blocage) alors on passe au case suivant, tout comme indiqué sur le réseau de pétri. Ce case représente le fait qu'un premier carton soit devant le capteur.

```
// Machine 2

switch (State_entity_2)
{

case 1: //transit_1_2
    if(Synch_1_2==1 && FACTORY_IO_Sensors_Get(entree_palettiseur) == 0)
    {
        State_entity_2 ++;
        Synch_1_2=0;
    }
    break;
```

- CASE 2

```
case 2://att carton 2
    if( FACTORY_IO_Sensors_Get(entree_palettiseur) == 1)
    {
        State_entity_2 ++;
    }
    break;
```

Ce case représente le fait que le premier carton ne soit plus devant le capteur, donc qu'il a avancé vers le blocage carton. Pour ce faire, on vérifie que le capteur "entree_palettiseur" soit repassé à 1 et si c'est le cas, on passe à l'entité suivante tel indiqué sur le réseau de Pétri.

- CASE 3

```
case 3: //attente_carton_2
    if(FACTORY_IO_Sensors_Get(entree_palettiseur) == 0)
    {
        State_entity_2 ++;
        Synch_2_3=1;
    }
    break;
```

Dans ce case, on met en application le fait d'attendre que le 2^e carton passe devant le capteur. Cette information va nous aider à savoir si l'on baisse ou non le blocage palettiseur dans le case suivant.

Ainsi, si l'on a bien le capteur "entree_palettiseur" == 0, alors on passe au case suivant et on passe à 1 la synchronisation entre l'entité 2 et 3. Cette synchro est tel que l'on souhaite vérifier que le capteur "butee_c" == 0 seulement quand l'on a bien 2 cartons au niveau du blocage de l'entrée palettiseur.

- CASE 4

```
case 4: //deblocage entree palettiseur
    Synch_3_2=0;
    buffer_Actuator &= ~(blocage_entree_pal);
    break;
}
```

Ici, on débloque l'entrée palettiseur et on passe à 0 la synchro entre l'entité 3 et 2 comme sur le réseau de Pétri. Cette synchro se fait quand le capteur « butée_c » passe à 0, (i.e qu'il a capté les cartons avant de les pousser) et ainsi on peut recommencer l'opération lorsque d'autres paires de cartons arriveront.

2.3.3 Entité 3 : poussoir cartons

Cette entité représente le poussoir, donc le mécanisme qui va pousser les paires de cartons sur la porte de l'ascenseur :

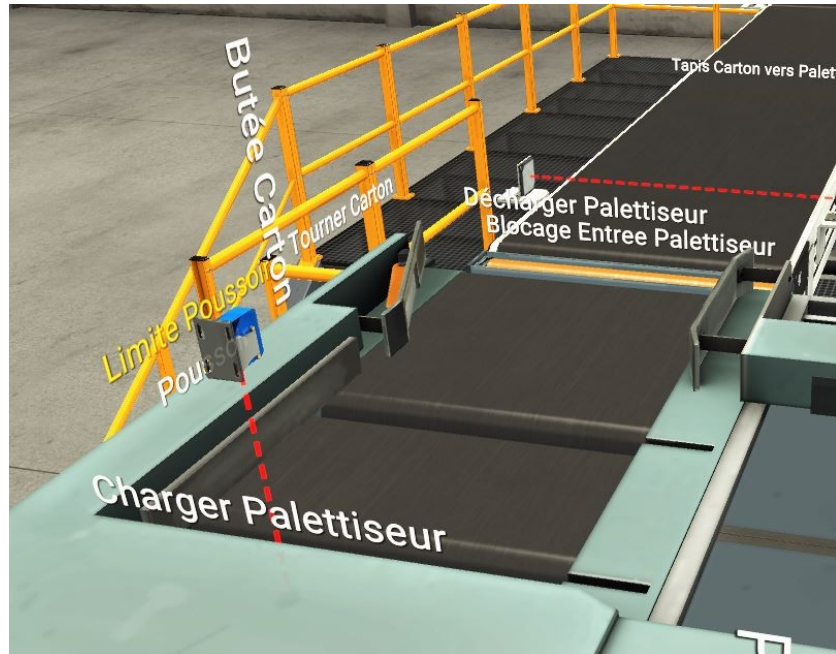


Figure 3: Entité 3 palettiseur

- CASE 1

Dans ce case, on attend d'avoir une synchronisation entre l'entité 2 et 3, quand c'est le cas et si le capteur "butee_c" capte quelque chose (==1), bien on va dans le case suivant et on passe à 1 la synchro 3_2.

```
// Machine 3
switch (State_entity_3)
{
    case 1://Att carton
        if(Synch_2_3==1 && FACTORY_IO_Sensors_Get(butee_c) == 1)
        {
            State_entity_3 ++;
            Synch_3_2=1;
        }
        break;
```

- CASE 2

Comme le capteur "butee_c" est passé à 1, on va pousser les cartons

```
case 2: //Poussoir
    buffer_Actuator |= poussoir;
    if(FACTORY_IO_Sensors_Get(limite_poussoir) == 1)
    {
        compteur_poussoir ++;
        State_entity_3 ++;
    }
    break;
```

grâce à la commande “buffer_Actuator |= poussoir”. Suite à cela, on incrémente de 1 un compteur afin de mettre en place un anneau mémoire à 3. Puis on change de case.

- CASE 3

Tant que la variable “compteur_poussoir” ne vaut pas 3, on retourne dans le case 1 afin de repousser des paires de cartons jusqu’à ce qu’il y en est trois.

Une fois que cela est fait on passe à 1 la synchronisation 3_4 afin de pouvoir gérer la porte dans la prochaine entité.

```
case 3: // attente_triple poussée
    if(compteur_poussoir==1)
    {
        State_entity_3=1;
    }
    else if(compteur_poussoir==2)
    {
        State_entity_3=1;
    }
    else if(compteur_poussoir==3)
    {
        State_entity_3=1;
        compteur_poussoir=0;
        Synch_3_4=1;
    }
    break;
}
```

2.3.4 Entité 4: Porte

Cette entité représente la porte de l’ascenseur, qui va s’ouvrir pour poser à chaque fois trois paires de cartons sur les palettes :

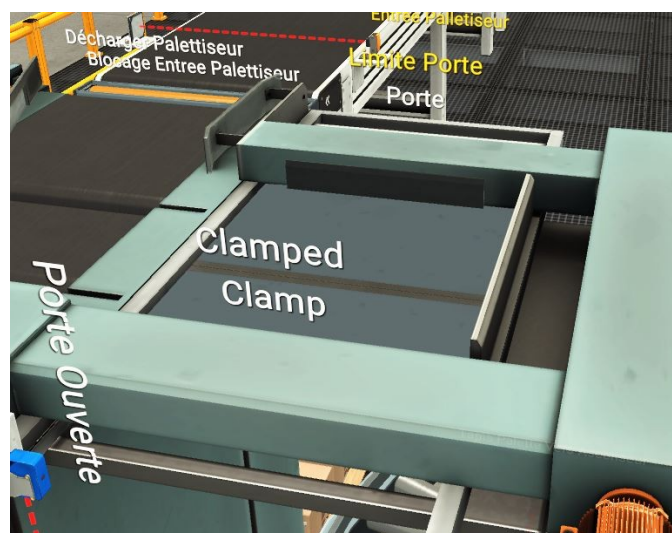


Figure 4: Entité 4 palettiseur

- CASE 1

Si on a eu 3 poussées de paires de cartons, alors on passe au case suivant.

```
// Machine 4
switch (State_entity_4)
{
case 1:
    if(compteur_poussoir == 3)
    {
        State_entity_4++;
        compteur_poussoir = 0;
    }
    break;
```

- CASE 2

On « clamp » les cartons grâce à la commande “buffer_Actuator | clamp” et on passe au case suivant. Cela signifie qu’on les range correctement sur la porte.

```
case 2:
    buffer_Actuator |= clamp;
    State_entity_4++;
    break;
```

- CASE 3

Une fois les cartons rangés, on peut ouvrir la porte et ainsi déposer les 6 cartons sur la palette.

```
case 3:
    if(FACTORY_IO_Sensors_Get(limite_porte) == 1 )
    {
        buffer_Actuator |= ouverture_porte;
    }
    break;
```

- CASE 4

On recommence ensuite le processus à l’infini dès que la porte se referme.

```
case 4:
    if(FACTORY_IO_Sensors_Get(limite_porte) == 1)
    {
        State_entity_4=1;
    }
    break;
}
```

2.3.5 Entité 5: Ascenseur



Figure 5: Entité 5 palettiseur

- CASE 1

On envoie des palettes grâce à la formule “buffer_Actuator |= distrib_pal” puis on passe au case suivant comme dans le réseau de Pétri.

```
// Machine 5
switch (State_entity_5)
{
case 1:
    buffer_Actuator |= (distrib_pal);
    State_entity_5++;
    break;
```

- CASE 2

Si le capteur “entree_pal” vaut 1 (i.e la palette arrive dans le système d’ascenseur) alors on charge la palette sur le système d’ascenseur. On attend ensuite que le capteur « sortie_pal » capte la palette. On peut donc passer au case suivant car cela veut dire que la palette est prête à monter.

```
case 2:
    if(FACTORY_IO_Sensors_Get(entree_pal) == 1)
    {
        buffer_Actuator |= (charger_pal);
    }
    if(FACTORY_IO_Sensors_Get(sortie_pal) == 1)
        State_entity_5++;
    break;
```

- CASE 3

Si la palette se trouve au niveau du capteur “sortie_pal” alors on passe à la suite.

```
case 3:
    if(FACTORY_IO_Sensors_Get(sortie_pal) == 1)
    {
        State_entity_5++;
    }
    break;
```

- CASE 4

Ici on monte jusqu'au 1er étage afin de réceptionner les cartons sur la palette.

```
case 4:
    buffer_Actuator |= (ascenseur_to_limit ) | (monter_ascenseur);
    State_entity_5++;
    break;
```

- CASE 5

Si le capteur « asc_1 » passe à 1, alors on arrête de faire monter la palette et on ouvre la porte afin d'y déposer des cartons (on n'a pas eu le temps de réaliser correctement la synchro entre l'entité 5 et 4).

```
case 5:
    if (FACTORY_IO_Sensors_Get(asc_1) == 1)
    {
        State_entity_5++;
        buffer_Actuator &= ~(monter_ascenseur);
        Synch_5_4=1; //ouverture de la porte cf machine 4
    }
```

- CASE 6

Une fois que l'on a déposé les cartons sur la palette, on fait descendre la palette afin de la faire partir vers la fin du circuit (cette partie n'a pas pu être terminée par manque de temps et d'organisation de notre part).

```
}
case 6:
    buffer_Actuator |= (ascenseur_to_limit ) | (descendre_ascenseur);
    State_entity_5=1;
}
```

3 Annexe

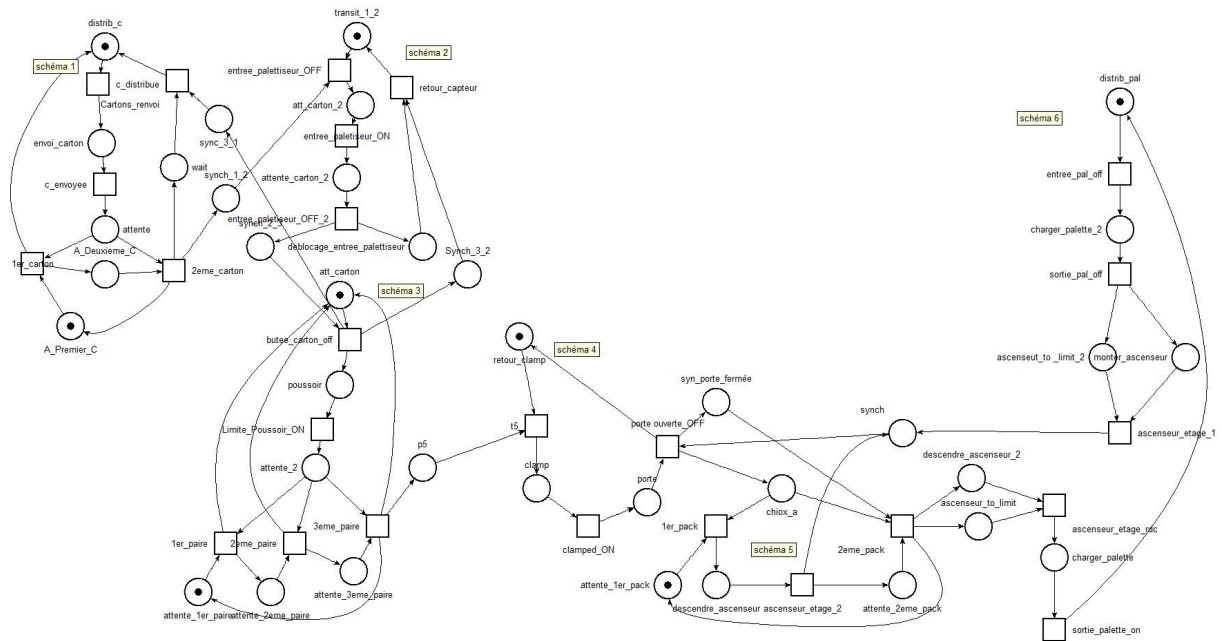


Figure 6: Réseau de pétri

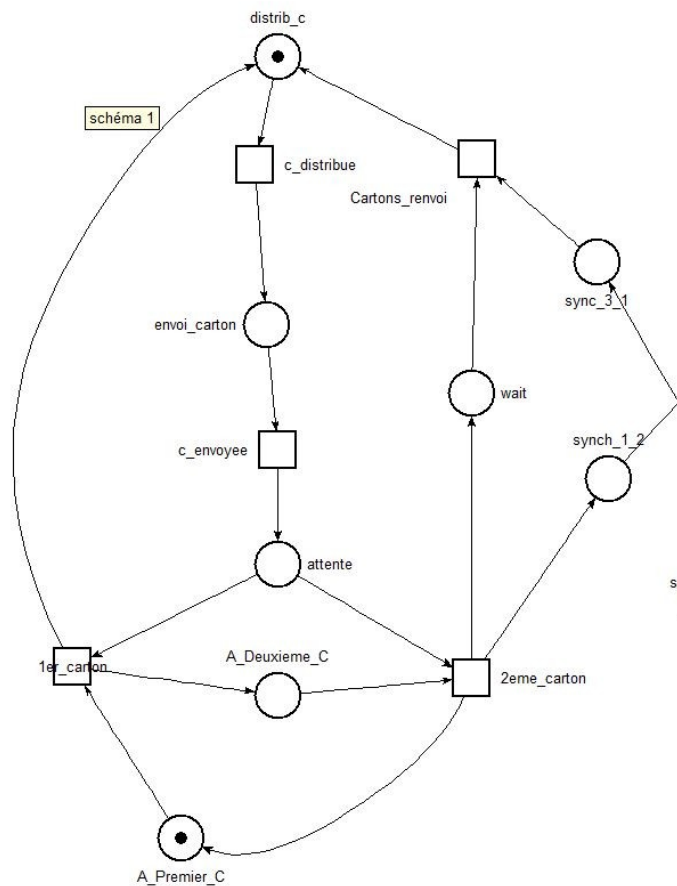


Figure 7: Entité 1

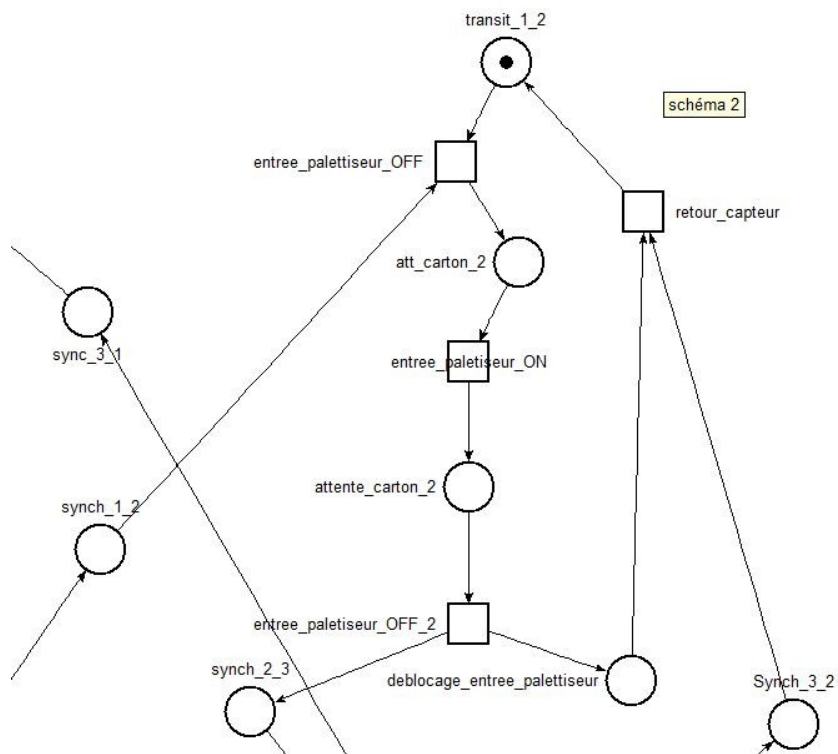


Figure 8: Entité 2

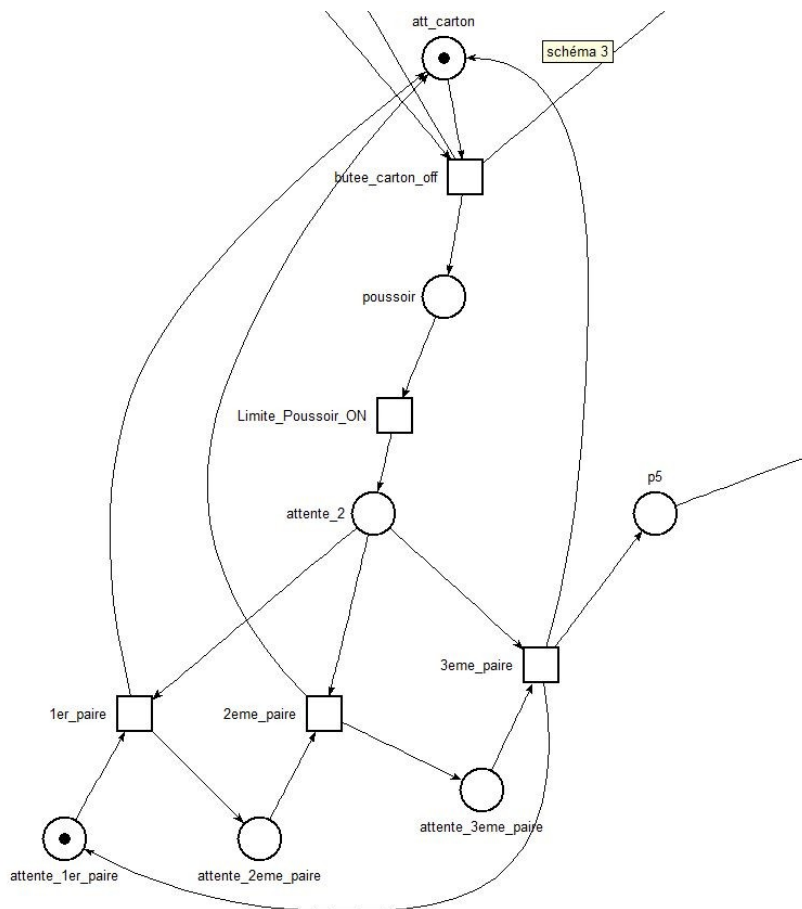


Figure 9: Entité 3

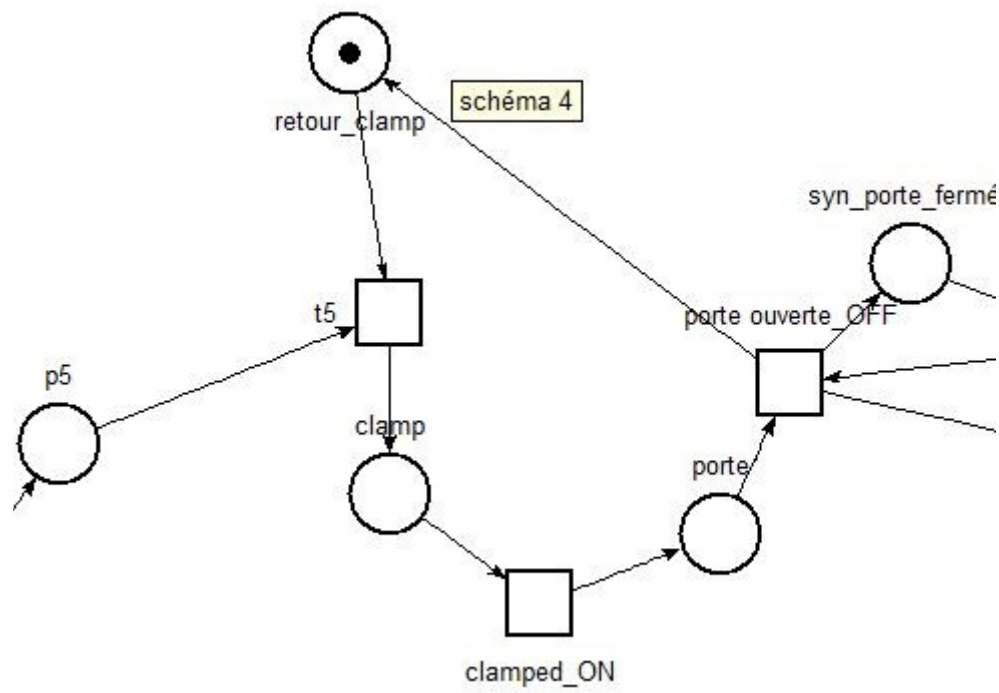


Figure 10: Entité 4

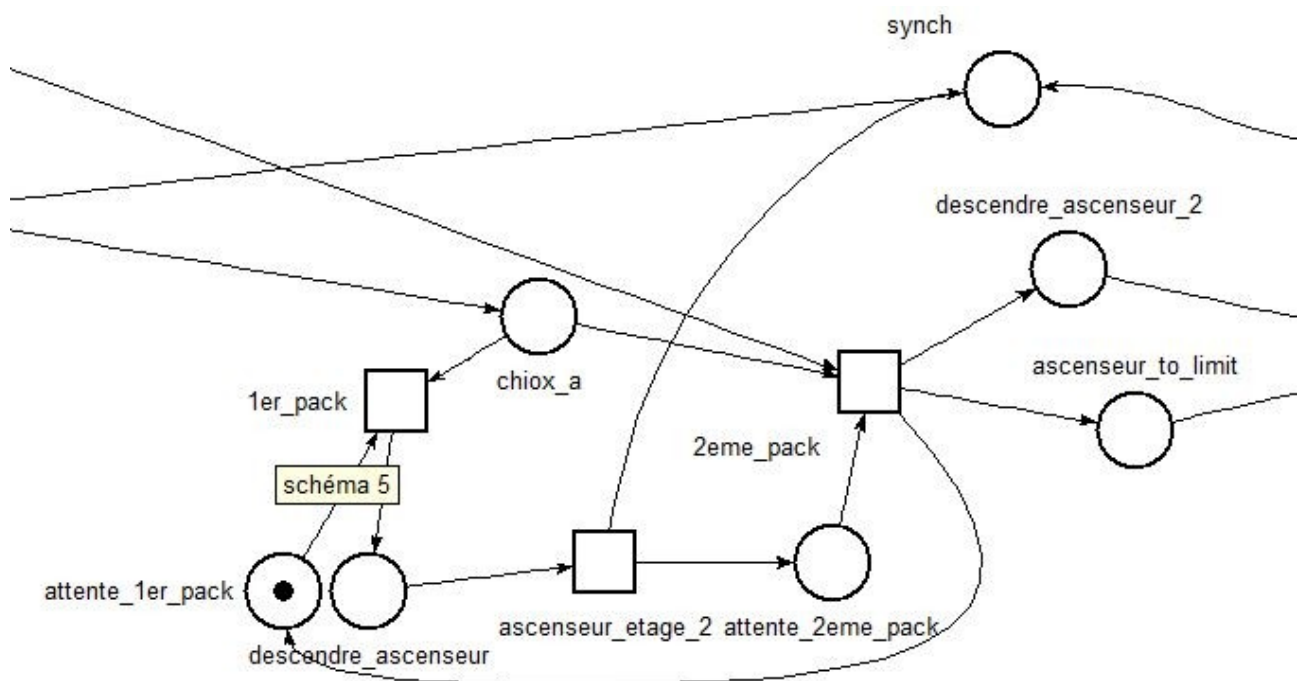


Figure 11: Entité 6

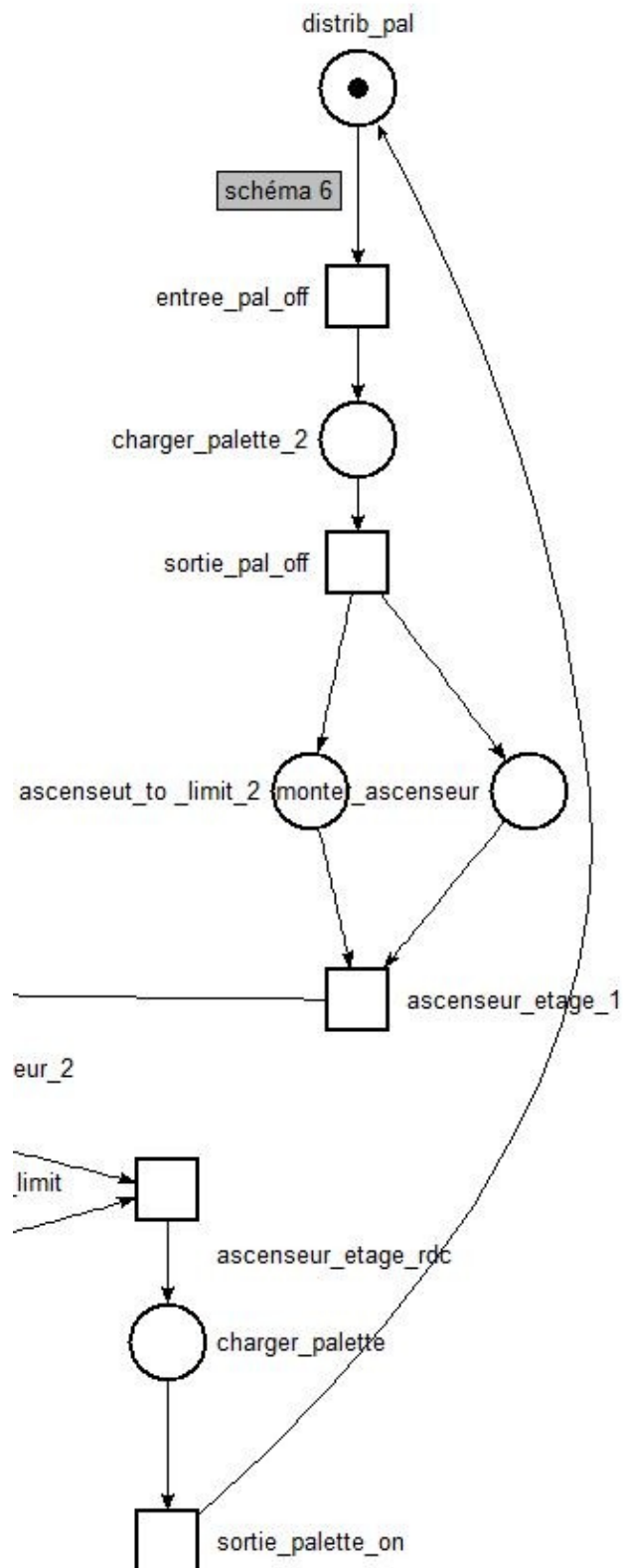


Figure 12: Entité 5